

Анализ журнала сессий средствами стека

Цель работы: Самостоятельно реализовать стек и применить его к разбору журнала событий входа и выхода: проверить корректность вложенности, обнаружить аномалии, собрать статистику. Оценить сложность операций, обеспечить устойчивость к повреждённому и враждебному вводу и на практике найти границу применимости стека как модели.

Теоретические сведения

Стек

Стек — линейная структура данных, работающая по принципу «последним пришёл — первым ушёл» (LIFO). Определены три операции: положить элемент на вершину (push), снять элемент с вершины (pop) и посмотреть вершину, не снимая (top). Все три выполняются за константное время, если стек реализован на динамическом массиве или связанном списке.

Стек естественно возникает всюду, где нужно отслеживать вложенность: сопоставление открывающих и закрывающих скобок, разбор выражений, стек вызовов функций, откат операций. Общий признак таких задач — «последнее открытое закрывается первым».

В этой работе запрещено использовать готовую реализацию стека из стандартной библиотеки (stack, Deque, list как стек и подобные). Стек реализуется самостоятельно. Остальные средства стандартной библиотеки разрешены.

Задача о сбалансированности

Классическое применение стека — проверка правильности вложенных скобок. Открывающая скобка кладётся на стек; закрывающая обязана совпасть с вершиной, иначе последовательность неправильная. В конце стек должен быть пуст.

Журнал сессий устроен так же. Событие входа LOGIN играет роль открывающей скобки, событие выхода LOGOUT — закрывающей. Корректный журнал — это правильно вложенная скобочная последовательность, где каждая закрывающая скобка совпадает с самой поздней ещё не закрытой открывающей.

Модель вложенных сессий и её граница

На базовом уровне сессии считаются строго вложенными: LOGOUT закрывает самую позднюю открытую сессию, как закрывающая скобка. Это удобная модель, но именно модель, а не описание реальности.

В настоящей системе сессии не вложены. Один пользователь может войти раньше другого и выйти позже: их интервалы пересекаются, хотя ничего некорректного не произошло. Стек на таком журнале начнёт сообщать о ложных нарушениях порядка, потому что модель вложенности к пересекающимся интервалам неприменима.

Обнаружить эту границу и перейти за неё — цель повышенного уровня. Одного стека там уже недостаточно; потребуется отдельный стек (или счётчик) на каждого пользователя, то есть словарь стеков. Так работа подводит к теме хеш-таблиц.

Сложность и устойчивость

Обработка журнала из n событий занимает время $O(n)$: каждое событие обрабатывается за константу, поскольку операции стека константны. Память в среднем определяется текущей

глубиной вложенности, а в худшем случае линейна по n — например, если журнал состоит из одних входов без выходов, на стек лягут все n записей.

Худший случай здесь не абстракция, а вектор атаки. Журнал из миллионов вложенных входов подряд — это либо переполнение стека вызовов у наивной рекурсивной реализации, либо исчерпание памяти. Разбор чужого, возможно испорченного, журнала — типичная задача аудита, поэтому программа обязана переживать любой вход: пустой файл, обрезанный на середине, строку в мегабайт, чужеродные байты, миллионы строк.

Транзакции аудита

Отдельный сюжет, важный для специальности, — семантика отката. В журналах аудита откат транзакции не стирает уже записанные события: запись о том, что действие было предпринято и затем отменено, сама по себе является частью аудиторского следа. Это отличается от отката в базе данных, где состояние возвращается к исходному. Точный контракт BEGIN/COMMIT/ROLLBACK — в разделе «Формат для повышенного уровня».

Формат ввода-вывода

Программа запускается из командной строки с двумя аргументами: путь к входному файлу и путь к выходному файлу. Диалога с пользователем быть не должно.

```
solution вход.txt выход.txt
```

Вход

Текстовый файл в кодировке UTF-8, одно событие на строку:

```
LOGIN <имя>  
LOGOUT <имя>
```

Имя пользователя — непустая последовательность букв и цифр без пробелов. Пустые строки и лишние пробелы между токенами допустимы и игнорируются. Любая другая строка считается некорректным событием.

Правила обработки (базовый уровень)

Состояние — стек имён. Журнал читается построчно:

Ситуация	Действие
LOGIN x	положить x на стек
LOGOUT x , вершина стека равна x	снять со стека, засчитать завершённую сессию
LOGOUT x , стек непуст, вершина не x	ошибка <code>order_violation</code> , стек не менять
LOGOUT x , стек пуст	ошибка <code>logout_without_login</code>
строка не является событием	ошибка <code>bad_event</code>
конец файла, стек непуст	по <code>session_not_closed</code> на каждую открытую сессию

Незакрываемые сессии выводятся в порядке возрастания номера строки, где они были открыты.

Выход

Сначала строки ошибок в порядке их обнаружения, по одной на строку:

```
ERROR <номер_строки> <код>
```

Код — один из: `order_violation`, `logout_without_login`, `bad_event`, `session_not_closed`. Номер строки считается с единицы. Последняя строка вывода — число завершённых сессий:

```
SESSIONS <n>
```

Сравнение вывода при проверке — потоковое по токенам: конкретные пробелы и переводы строк значения не имеют, важны сами токены и их порядок.

Пример

Вход:

```
LOGIN alice
LOGIN bob
LOGOUT bob
LOGOUT alice
LOGOUT carol
```

Выход:

```
ERROR 5 logout_without_login
SESSIONS 2
```

Разбор тонкости. При входе «`LOGIN alice`, `LOGIN bob`, `LOGOUT alice`» третья строка даёт `order_violation`, потому что на вершине стека находится `bob`, а не `alice`; при этом стек не изменяется. Поэтому последующий `LOGOUT bob` штатно закрывает сессию `bob`, а сессия `alice` остаётся незакрытой и в конце даёт `session_not_closed`. Ошибка `order_violation` не снимает вершину — это частая ошибка реализации.

Формат для повышенного уровня

Базовый формат событий и коды ошибок — как в разделе «Формат ввода-вывода» выше. Здесь описано только то, что меняется или добавляется по каждому расширению.

Пересекающиеся сессии

Формат входа не меняется — по-прежнему `LOGIN <имя>` / `LOGOUT <имя>`. Меняется модель: `LOGOUT x` закрывает открытую сессию именно пользователя `x` (словарь стеков по пользователям, у каждого — свой стек), а не вершину одного общего стека. Как следствие, `order_violation` в этой модели структурно невозможен: в стек пользователя `x` по построению попадают только его собственные `LOGIN`, поэтому вершина всегда совпадает с `x`. Проверьте это утверждение сами, прежде чем полагаться на него в отчёте.

Статистика активности

Пиковое число одновременно активных сессий и момент его достижения считаются по обычным `LOGIN/LOGOUT`, без изменения формата входа. «Момент» — номер строки, на которой пик впервые достигнут; при нескольких строках с одинаковым пиком — самая ранняя из них.

Отметки времени (для средней длительности сессии)

Если добавляете это подрасширение, строка события расширяется третьим токеном — целым неотрицательным числом тактов:

```
LOGIN <имя> <время>
LOGOUT <имя> <время>
```

Время монотонно не убывает по номеру строки (события идут в хронологическом порядке; несколько событий могут иметь одинаковое время). Это отдельный диалект формата, а не дополнение к базовому: строка ровно из двух токенов в нём уже не встречается — в файле либо везде два токена (базовый формат), либо везде три (этот вариант), смешивать нельзя.

Средняя длительность сессии — среднее арифметическое (время `LOGOUT` – время `LOGIN`) по всем завершённым сессиям; на пустом множестве завершённых сессий не определена — решите сами и опишите в отчёте, что в этом случае печатает программа.

Транзакции аудита

К `LOGIN/LOGOUT` добавляются три команды без аргументов:

- BEGIN — открыть транзакцию: запомнить снимок текущего стека сессий.
- COMMIT — закрыть транзакцию, зафиксировав всё, что произошло внутри.
- ROLLBACK — закрыть транзакцию, вернув стек сессий к снимку на момент BEGIN.

Уровень базовый: один общий стек сессий (как в базовом уровне работы), не словарь стеков по пользователям — расширения независимы и одновременно не предполагались.

ROLLBACK откатывает только стек открытых сессий. Никогда не откатываются: уже выведенные ошибки (часть аудиторского следа) и счётчик SESSIONS (накопленная статистика, а не состояние). Следствие: сессия, открытая до BEGIN и закрытая внутри транзакции, при ROLLBACK «воскресает» — возвращается на стек как открытая, хотя её закрытие уже засчитано в SESSIONS; сессия, открытая внутри транзакции, при ROLLBACK пропадает со стека целиком, как будто не открывалась.

Новые коды ошибок (ERROR <строка> <код>, как в базовом формате):

- nested_begin — BEGIN при уже открытой транзакции; вложенный BEGIN игнорируется, активна первая транзакция.
- no_transaction — COMMIT или ROLLBACK без открытой транзакции; команда игнорируется.
- transaction_not_closed — конец файла при открытой транзакции; ошибка на строке BEGIN, трактуется как неявный COMMIT (открытое остаётся открытым и даёт обычный session_not_closed).

Пример:

```
LOGIN alice
BEGIN
LOGIN bob
LOGOUT bob
ROLLBACK
LOGOUT alice
```

bob целиком открылся и закрылся внутри транзакции — ROLLBACK стирает его так, будто не было; alice транзакции не касалась. Итог: SESSIONS 2, ошибок нет (закрытие bob всё равно учтено в счётчике, несмотря на откат).

Важный нюанс порядка вывода: ошибки времени сканирования (order_violation, logout_without_login, bad_event, nested_begin, no_transaction) печатаются строго в порядке обнаружения при проходе сверху вниз. Ошибки конца файла (transaction_not_closed, session_not_closed) печатаются отдельным блоком после всех ошибок сканирования, отсортированным по номеру строки только внутри себя — не единой сортировкой всех ошибок по номеру строки.

Реализация: стек имён, флаг «транзакция открыта» и глубина стека на момент BEGIN (снимок). Одной глубины мало: сессии, закрытые внутри транзакции и лежавшие ниже снимка, нужно не просто подсчитать, а вернуть на стек в исходном порядке при откате — держите отдельный буфер снятых имён (LIFO) и возвращайте его в обратном порядке накопления на ROLLBACK.

Базовые требования к программе

1. Стек реализован самостоятельно; готовые контейнеры-стеки стандартной библиотеки не используются.
2. Журнал обрабатывается за один проход, время линейно по числу событий.
3. Все четыре вида аномалий обнаруживаются и выводятся в указанном формате.
4. Программа запускается из командной строки с фиксированными аргументами и не требует ввода с клавиатуры.
5. Программа не завершается аварийно ни на каком входном файле, включая пустой, усечённый, чрезмерно большой и содержащий чужеродные байты. Допустим аккуратный выход с сообщением об ошибке; недопустимы аварийное завершение и зависание.

6. Предусмотрен режим или отдельная сборка для замера времени обработки, отделяющий чтение файла от собственно разбора.

Перед сдачей работа прогоняется через проверочный скрипт репозитория:

```
python3 ../check.py r1-session-log
```

Зелёный результат по публичным тестам необходим, но не достаточен: при приёме добавляются скрытые тесты на краевые случаи, не описанные в задании.

Индивидуализация и честность работы

Базовая постановка одинакова для всей группы — это позволяет пользоваться общими публичными тестами и взаимной проверкой. Индивидуальность работы обеспечивается тремя способами:

- Данные для замеров и отчёта генерируются под каждого студента отдельным сидом; числа в вашем отчёте не совпадут с чужими.
- Каждый студент сдаёт входной файл, ломающий или существенно замедляющий программу однокурсника, с указанием автора. Найденная поломка засчитывается автору теста, починенная — автору программы.
- Выбор расширения на повышенном уровне остаётся за студентом.

Совпадение вывода вашей программы с чужим на индивидуальных данных — маркер списывания.

Дополнительное задание (повышенный уровень)

Обязателен для выхода на отличную оценку. Первый пункт основной; из остальных выбрать не менее одного.

- Пересекающиеся сессии. Обработать настоящий журнал, где сессии не вложены, а пересекаются. Показать, что одного стека недостаточно, и реализовать словарь стеков (или счётчиков) по пользователям. В отчёте объяснить, на каком входе наивная модель со стеком начинает выдавать ложные `order_violation` и почему словарь стеков это устраняет. Формат входа при этом не меняется — подробности в разделе «Формат для повышенного уровня».
- Явный стек против рекурсии. Если базовое решение использует рекурсию, переписать обход на явный стек и предъявить вход, на котором рекурсивная версия исчерпывает стек вызовов, а итеративная работает. Оценить предельную глубину для обеих версий.
- Статистика активности. Вычислить число одновременно активных сессий во времени: пиковое значение и момент его достижения; при наличии временных отметок — среднюю длительность сессии. Формат отметок времени и точное определение «момента» — в разделе «Формат для повышенного уровня».
- Транзакции аудита. Реализовать `BEGIN` / `COMMIT` / `ROLLBACK` по контракту из раздела «Формат для повышенного уровня», с сохранением семантики аудита: откат не стирает уже зафиксированные события об ошибках. В отчёте обсудить, чем это отличается от отката в базе данных и почему для аудита выбрана именно такая семантика.

Требования к отчёту

Отчёт оформляется по общему шаблону курса (`report-template.md`) и содержит обязательные разделы паспорта работы.

1. Асимптотика по времени и памяти с обоснованием. Оценку памяти дать в двух случаях: сбалансированный журнал и журнал из одних входов без выходов.
2. Замеры. Для этой работы раздел облегчён: снимите время обработки на входах растущего размера любым доступным способом. Это первая работа курса, и полноценный протокол измерений ещё не введён — ваши наивные замеры станут материалом для семинара по методологии измерений.
3. Худший случай: из чего он возникает (глубина вложенности), вход, который его вызывает, и что при этом происходит с памятью или стеком вызовов.
4. Масштабирование: что откажет первым при росте журнала на три порядка.
5. Граница модели: на каком входе вложенная модель ломается; если выполнен повышенный уровень — как словарь стеков её снимает.
6. Устойчивость: какой враждебный тест вы сдали против чужой программы и какие атаки на вашу программу вы отразили.

Контрольные вопросы

1. Почему стек — естественная структура для проверки вложенности? Какой признак задачи на это указывает?
2. Почему ошибка `order_violation` не должна снимать вершину стека? Что сломается, если снимать?
3. Оцените время и память вашей программы. В каком случае память максимальна?
4. Приведите вход, на котором рекурсивная обработка исчерпает стек вызовов. Как это связано с худшим случаем по памяти?
5. Почему модель вложенных сессий неприменима к реальному журналу? Приведите конкретный входной пример с пересечением.
6. Как устроен переход от одного стека к словарю стеков? Что становится ключом словаря?
7. Чем откат транзакции в журнале аудита отличается от отката в базе данных и почему?
8. Какие классы некорректного ввода вы предусмотрели и как программа на них реагирует?